# HabitFlow

Owner A — Core Product & Foundation

Product Requirements Document + AI Build Specification

Self-contained implementation brief derived from the master HabitFlow BRD and build specification. Shared requirements apply to both collaborators; ownership boundaries identify the work assigned to this document.

### Implementation goal

Build the stable shared foundation and core tracking experience. Your work defines the contracts your friend’s modules consume.

### Collaboration rule

Use one shared GitHub repository. Each person works on a feature branch and opens a pull request. Keep main buildable, agree shared interfaces before parallel work, and pull merged changes into the local Antigravity clone.

# How to use this document

This document includes the shared BRD requirements needed to understand the product, followed by this collaborator's implementation scope. The master combined PDF remains authoritative. Do not independently implement the other person's module unless both collaborators agree.

### Ownership boundary

Assigned owner: Owner A — Core Product & Foundation. Shared files such as database schema, global types, authentication, date/time utilities, app layout, and service-worker registration require coordination before edits. Avoid concurrent edits to the same files.

# 1. Product overview

HabitFlow is a private, customizable hourly habit and behavior tracking web app. Users create questions, answer them across time slots, and understand patterns through a spreadsheet-style Track Sheet and analytics.

Product philosophy: Track honestly → Understand patterns → Measure improvement → Improve gradually. Keep the product fast, private, minimal, professional, data-driven, non-judgmental, and easy to use repeatedly.

# 2. Critical answer/outcome rules

**Raw answer and evaluated outcome are separate.** Never assume YES means Positive or NO means Negative. Every option has its own configured Positive, Negative, or Neutral outcome.

Store the selected answer and an outcome snapshot on each entry. The Track Sheet can visualize the outcome; detail views preserve the original answer. Changing option rules must not silently rewrite history. Unanswered is missing data, never a negative result.

# 3. Shared UX and core flow

Navigation: Dashboard, Track Sheet, Questions, Analytics (Daily/Weekly/Monthly), Insights, Settings. The two screenshots in the master PDF are the visual source of truth. Preserve their recognizable layout, density, spacing, colors, typography, and interactions; do not replace them with a generic dashboard.

Track Sheet rows are questions; columns are time slots; each cell is one question/slot/date. Phase 1 default is 6 AM–9 PM. Selecting a cell opens an answer dialog; save updates the cell without full-page reload. Include date navigation and entry edit/delete.

Question management includes create/edit, category, ordering, activate/pause/archive/delete, configurable answer options, outcome mapping, and optional explanations.

# 4. Analytics and insights rules

Daily analytics: outcome counts, answered/unanswered slots, completion, and time-of-day summaries. Weekly analytics: seven-day comparison and change over time. Monthly analytics: trends and heatmap. Insights must come from real saved entries and explain the time window/sample size when practical.

Use explicit denominators; do not mislead with one score when questions have different meanings. Correlation does not prove causation. Never fabricate analytics or production sample values.

# 5. Security and reliability

Authentication: sign-up, login, logout, session persistence, protected routes, strict user-level data isolation. Every server-side read/write must authorize the current user; never trust client-supplied user IDs.

Validate inputs server-side. Keep secrets in environment variables; never expose private keys in browser code. Provide loading, empty, validation, error and success states. Persist data across refresh/navigation; report failed saves honestly.

Use modular components, strict TypeScript where practical, reusable calculations, migrations, and tests. Preserve compatible existing infrastructure and avoid unnecessary paid services.

# 6. Mobile-first and accessibility

Test widths 320, 360, 375, 390 and 430 CSS px plus tablet/desktop. Avoid accidental page overflow. Keep question labels visible while the hourly grid scrolls horizontally, or use a focused single-question mobile view; do not crush columns.

Use practical 44×44 px touch targets, keyboard access, clear labels/focus, sufficient contrast, responsive charts/tables/dialogs, safe-area spacing, and no essential hover-only controls.

## 7. Shared technical contract

Preferred stack when compatible: Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, PostgreSQL/Neon, Drizzle ORM, compatible auth such as Better Auth, Recharts, Vercel. Change only for a documented compatibility reason.

Shared entities: auth users/sessions; questions (id, user_id, text, category, answer_type, active, order, timestamps); question_options (id, question_id, text, outcome, explanation, order); hourly_entries (id, user_id, question_id, option/value, date, time slot, outcome_snapshot, timestamps); settings/reminder preferences as needed.

Before parallel development, agree TypeScript types, data-access/query functions, route shapes, timezone/date rules, shared file ownership, environment variables, and migration workflow. Never build against guessed database fields or create a second auth/database architecture.

## 8. Phases

Phase 1: foundation, auth/authorization, database, Yes/No question management and outcomes, Track Sheet, entry management, dashboard, daily/weekly/monthly analytics, basic insights, responsive polish and reliability.

Phase 2 is planned scope: custom tracking hours/intervals; single-choice, multiple-choice, numeric, currency, duration and rating questions; goals/streaks; CSV/PDF exports; advanced behavioral correlations; optional AI summaries only after basic analytics are reliable. Build extensible foundations now and implement Phase 2 in separate PRs after dependencies are ready.

## 9. Notifications and PWA

Support interval reminders and specific clock-time reminders, timezone/DST, configured quiet hours/weekdays, duplicate prevention, sensible missed reminders, subscription expiry/recovery, and navigation to Track Sheet. Avoid sensitive habit answers in push payloads by default.

Background reminders require supported service worker/Web Push and server-side scheduling. Browser timers alone cannot guarantee delivery after browser closure. Explain platform/permission limits accurately.

PWA: manifest, app name/icons, standalone display, HTTPS service worker, install action/manual instructions, installed-state handling, safe offline fallback and updates that preserve unsaved entries. Do not indiscriminately cache private authenticated APIs.

## 10. Definition of done

A feature is not complete just because the UI exists. Verify persistence, authorization, validation, loading/empty/error states, responsive behavior, tests and build/type-check/lint results. Include migrations and document blocked checks.

Inspect repository and Git status first. Work in focused branches; never commit secrets. Each PR explains purpose, files, screenshots for UI changes, tests, migration/environment needs, and unresolved issues.

# Assigned implementation scope

Build the stable shared foundation and core tracking experience. Your work defines the contracts your friend's modules consume.

## A1. Repository and app foundation

- Initialize the app in the shared GitHub repo; configure scripts, lint/type-check, base styles, shared layout/navigation, environment example, README, and local setup.
- Set folder structure, conventions, error handling and tests. Commit a minimal working scaffold before parallel feature development.

## A2. Authentication, database and security

- Implement sign-up/login/logout/session persistence and protected routes with the chosen compatible auth system.
- Create database connection/schema/migrations and server-side data-access layer. Enforce current-user ownership in every query/mutation; test cross-user access denial.
- Publish the shared integration contract: schema/types, query functions, route shapes, timezone/date rules and test data. Tell Owner B when stable.

## A3. Questions and outcome mapping

- Implement question CRUD, categories, display order, active/archive state, Yes/No options for Phase 1, explanations and independent Positive/Negative/Neutral mappings.
- Use extensible types for Phase 2 question formats. Coordinate schema changes before merge; do not silently change shared contracts.

## A4. Hourly Track Sheet

- Build the reference-led dense grid for 6 AM–9 PM, question rows, time columns, selected date/navigation, answer dialog, cell outcome display, and create/edit/delete entry flows.
- Give immediate feedback only after successful persistence, validate saves, recover from failures, and keep labels visible during mobile grid scrolling.

## A5. Dashboard and shared calculations

- Build basic dashboard and pure utilities for answered/unanswered counts, completion, outcome mapping, date boundaries, time slots and timezone-aware dates.
- Document calculations for Owner B. Other modules must consume these utilities rather than duplicate or redefine scoring/missing-data rules.

## A6. Phase 2 foundations

- Own custom tracking hours/intervals and extensible question-type data model where not already included in core. Preserve the 6 AM–9 PM default and existing entries when schedules change. Coordinate migrations before dependent modules begin.

## A7. Owner A acceptance

- Both laptops can clone and run from README; auth, persistence and user isolation are verified.
- Questions/options persist; YES/NO do not hardcode outcome semantics; unanswered is not negative.
- Track entries add/edit/delete, survive refresh and support date navigation. Shared contracts are documented; tests/build pass or blockers are explicit.
- UI follows reference screenshots and works at phone/tablet/desktop sizes.

# Implementation process and GitHub workflow

| Step | Required process |
|---|---|
| 1. Inspect before editing | Open the repository in Antigravity. Review README, package scripts, Git status and current code. Do not overwrite uncommitted work. |
| 2. Branch per feature | Update main and create a focused branch such as feat/track-sheet, feat/analytics, feat/reminders or feat/pwa. |
| 3. Agree interfaces | Before relying on schema or shared utilities, agree TypeScript types, query functions, API behavior, date/time rules and shared-file ownership. |
| 4. Implement and test | Add tests, run lint/type-check/build, test responsive states and loading/empty/error states. Report tests accurately. |
| 5. Commit and push | Review git diff; exclude secrets/local data; make a descriptive commit and push the feature branch. |
| 6. Pull request | Explain purpose, requirements, files changed, UI screenshots, migrations, environment changes, test results and blockers. |
| 7. Integrate locally | After merge, run git switch main and git pull origin main in Antigravity's terminal. Install changed dependencies and run migrations if needed. |
| 8. Resolve conflicts safely | Commit or stash local work before switching branches. Merge/rebase only using the agreed team workflow; resolve conflicts deliberately and rerun checks. |

## Common Git commands

```
git clone https://github.com/OWNER/REPOSITORY.git

cd REPOSITORY

git switch main

git pull origin main

git switch -c feat/your-feature

# after implementation and tests

git add .

git commit -m "feat: describe the feature"

git push -u origin feat/your-feature
```

To receive merged work on this laptop: commit or stash your changes, then run git switch main and git pull origin main. GitHub does not sync files into Antigravity automatically; pull updates the local clone.

# Acceptance criteria and release gate

- The UI follows the reference-led design and is responsive on mobile, tablet and desktop.
- All reads/writes are authenticated and scoped to the current user server-side.
- Raw answers and outcome snapshots remain separate; unanswered cells are not negative.
- Data persists across refresh/navigation and failed saves are reported.
- Analytics and exports use real data and documented rules.
- Push/PWA limitations are accurate; private data is not exposed through push payloads or unsafe caches.
- Relevant tests, lint, type-check and production build are run and actual results reported.
- No secrets are committed; migrations/environment changes are documented.
- Pull request is focused and does not silently change shared contracts.

# Antigravity prompt for this owner

Read the master HabitFlow Combined BRD + AI Build Specification and this Owner A PRD/build specification. Inspect repository and Git status before editing. My ownership: project foundation, auth/database/security, question management and outcome mapping, hourly Track Sheet, dashboard/shared calculations, and agreed custom schedule/question-type foundations. Make a plan and list files/interfaces/migrations before coding. Implement incrementally, preserve reference UI, coordinate shared contracts with Owner B, test and report exact changes/results/blockers. Do not implement Owner B modules without agreement; never hardcode YES as Positive, treat unanswered as Negative, expose secrets, or claim untested features work.

## Phase 2 scope reminder

Phase 2 is planned scope: custom tracking hours/intervals; single-choice, multiple-choice, numeric, currency, duration and rating questions; goals/streaks; CSV/PDF exports; advanced behavioral correlations; and optional AI summaries after basic analytics are reliable. Implement only the assigned slice and coordinate shared dependencies.